

Replicação AgentRx: Diagnóstico em Falhas de Sistemas Multiagentes utilizando Telemetria Estruturada

Pedro Henrique Silva Da Costa Gregório
pedro.gregorio@copin.ufcg.edu.br
Universidade Federal de Campina Grande (UFCG)
Campina Grande, PB, Brasil

Resumo

A depuração de Sistemas Multiagentes (MAS) baseados em Grandes Modelos de Linguagem (LLMs) é um problema relevante em *AgentOps*, pois esses sistemas operam de forma autônoma, não determinística e com múltiplos artefatos de execução ao longo da trajetória. O artigo AgentRx propõe um framework para diagnosticar falhas a partir dessas trajetórias, localizando a etapa crítica irreversível e classificando sua causa raiz a partir de restrições, evidências e julgamento via LLM. Entretanto, o estudo original depende de logs textuais heterogêneos, o que exige normalização complexa para uma representação intermediária. Neste trabalho, propomos uma replicação do AgentRx baseada em um MAS simulado e instrumentado com telemetria baseada no OpenTelemetry, comparando logs textuais simplificados e traces estruturados para avaliar a precisão do diagnóstico e da classificação de falhas.

Keywords

Sistemas Multiagentes, AgentOps, OpenTelemetry, Telemetria Estruturada, Injeção de Falhas, Diagnóstico de Falhas

ACM Reference Format:

Pedro Henrique Silva Da Costa Gregório. 2018. Replicação AgentRx: Diagnóstico em Falhas de Sistemas Multiagentes utilizando Telemetria Estruturada. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 7 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Introdução

Sistemas Multiagentes (do inglês *Mult Agent systems*, MAS) baseados em Grandes Modelos de Linguagem (LLMs) vêm sendo empregados em tarefas cada vez mais complexas, combinando planejamento, uso de ferramentas e coordenação entre agentes. Essa autonomia, embora benéfica do ponto de vista funcional, amplia a dificuldade de observação e depuração, pois as execuções são probabilísticas, os fluxos são longos e os artefatos gerados ao longo da trajetória são variados [2]. Nesse contexto, AgentOps surge como um paradigma de observabilidade voltado a rastrear, correlacionar

e analisar os artefatos produzidos por agentes ao longo de seu ciclo de vida, incluindo agente, plano, workflow, tarefa, ferramenta, avaliação e guardrails [2].

Os artigos recentes tem mostrado que a observabilidade de agentes não deve se limitar ao resultado final da tarefa, mas também ao processo de execução que leva a esse resultado. O estudo de Dong et al. [2] destaca que agentes de IA produzem artefatos complexos em diferentes camadas, e que esses artefatos devem ser rastreados por meio de traces, spans, eventos e métricas para permitir diagnóstico e responsabilização.

Dentro desse cenário, o AgentRx propõe um framework de diagnóstico para localizar a primeira falha irreversível em trajetórias de agentes e atribuir sua causa raiz. Em sua formulação original, o método foi avaliado sobre 115 trajetórias falhas coletadas em três domínios reais e heterogêneos (*tau-bench*, *Flash* e *Magentic-One*), anotadas manualmente com a etapa crítica e a categoria da falha [1]. O fluxo do framework normaliza os rastros em uma Representação Intermediária (IR), sintetiza restrições globais e dinâmicas, produz um log de validação auditável e utiliza um juiz baseado em LLM para inferir a etapa crítica e a categoria da falha [1].

Ainda conforme o estudo, os autores apontam que o AgentRx pode ser induzido ao erro quando os sinais de validação são fracos ou contêm falsos positivos, e indicam como trabalho futuro a identificação de conjuntos de sinais de maior qualidade, capazes de separar verdadeiros positivos de sinalizações ruidosas [1]. Se a qualidade do diagnóstico depende da qualidade dos sinais disponíveis na trajetória, então a forma como a execução é registrada e representada torna-se determinante. Grande parte dos sistemas de agentes registra sua execução por meio de logs textuais, que tendem a ser heterogêneos, incompletos ou ambíguos, exigindo etapas adicionais de normalização antes de qualquer análise. Uma representação estruturada da trajetória, por outro lado, pode tornar explícitos elementos que no texto ficam implícitos como os agentes, ações, ferramentas, eventos, status, erros e as relações hierárquicas entre operações.

Nesse ponto, o OpenTelemetry se apresenta como um candidato a fonte de sinais estruturados, por fornecer um vocabulário padronizado para representar execuções observáveis, incluindo atributos de spans, relações hierárquicas entre operações, eventos e métricas quantitativas associadas a custo, duração e uso de tokens [3]. Não está claro, entretanto, se essa estruturação de fato melhora o diagnóstico produzido pelo AgentRx frente a representações textuais mais simples: é possível que os sinais estruturados reduzam a ambiguidade, mas também que apenas acrescentem informação que o juiz não aproveita. Investigar essa questão de forma controlada é o objetivo deste trabalho.

Para isso, este estudo realiza uma replicação controlada do AgentRx em um ambiente simulado de Sistemas Multiagentes, no qual as trajetórias são geradas experimentalmente e representadas de duas formas alternativas: logs textuais simplificados e telemetria estruturada inspirada em OpenTelemetry. Sobre os mesmos cenários pareados, com falhas injetadas de categorias conhecidas, comparam-se as duas representações para examinar tanto a replicabilidade do diagnóstico em um novo contexto quanto o efeito da estruturação dos registros sobre a atribuição de falhas.

As questões de pesquisa que guiam este estudo são:

- **RQ1:** A telemetria estruturada melhora a identificação da etapa crítica da falha em comparação com logs textuais?
- **RQ2:** A telemetria estruturada reduz ambiguidades na categorização da causa raiz das falhas?

As principais contribuições deste trabalho são: (i) uma adaptação experimental do AgentRx para um MAS simulado com falhas injetadas de categorias conhecidas; (ii) uma comparação pareada entre uma representação textual simplificada e uma representação com telemetria estruturada da mesma trajetória, ambas processadas pelo AgentRx; (iii) uma análise exploratória do efeito da representação da trajetória sobre a localização da etapa crítica e a classificação da categoria da causa raiz; e (iv) uma discussão sobre limitações e sobre quais sinais de telemetria podem apoiar diagnósticos futuros.

Neste contexto, este trabalho está organizado seguindo a estrutura: A Seção 2 apresenta o AgentRx, o modelo de telemetria proposto e a integração entre ambos. A Seção 3 descreve o desenho experimental, a instrumentação, as métricas e a análise estatística planejada.

2 Fundamentação Teórica

O AgentRx é um framework automatizado e independente de domínio para diagnóstico de execuções agênticas. Seu objetivo é localizar a *falha crítica*, definida como a primeira falha irrecuperável ao longo da trajetória, e atribuir sua causa raiz [1]. A proposta responde à dificuldade de depurar sistemas agênticos, nos quais execuções probabilísticas, longas, multiagente e dependentes de ferramentas podem fazer com que erros se propaguem antes de se tornarem observáveis [1]. Para avaliar o método, os autores construíram um *benchmark* com 115 trajetórias falhas, anotadas manualmente com etapa crítica e categoria de falha, provenientes de três domínios: fluxos de API estruturados (*tau-bench*), gerenciamento de incidentes (*Flash*) e tarefas abertas envolvendo web e arquivos (*Magentic-One*) [1].

Operacionalmente, o AgentRx recebe uma trajetória falha, um conjunto de ferramentas com seus esquemas de entrada e saída e, opcionalmente, uma política de domínio em linguagem natural. Como os formatos de execução variam entre domínios, a trajetória é primeiro normalizada em uma Representação Intermediária (IR) comum [1]. Em seguida, o framework sintetiza restrições em dois níveis: restrições globais, derivadas do esquema das ferramentas e da política de domínio, e restrições dinâmicas, geradas a cada etapa a partir da instrução da tarefa e do prefixo da trajetória observado até aquele ponto. Cada restrição combina uma guarda, que

define quando ela se aplica, e uma asserção, que produz um veredito binário por meio de verificações programáticas ou semânticas avaliadas por LLM [1].

A avaliação das restrições produz um log de validação indexado por etapa, que registra cada restrição violada e sua evidência de suporte [1]. Esse log, juntamente com a instrução da tarefa, a trajetória e uma lista de verificação (*checklist*) semântica derivada da taxonomia, é fornecido a um juiz baseado em LLM, que infere a etapa crítica e a categoria da falha [1]. Segundo os autores, esse acoplamento entre síntese de restrições e adjudicação por LLM apoiada em evidências melhora a localização da etapa e a atribuição da causa raiz em relação a linhas de base existentes nos três domínios avaliados [1].

Uma das contribuições do AgentRx é uma taxonomia de falhas multidomínio, obtida por um processo de codificação por *grounded theory*, no qual as categorias emergiram de padrões recorrentes de falha em vez de serem impostas a priori [1]. A taxonomia rotula a causa raiz da falha crítica e é composta por nove categorias:

- (1) **Instruction/Plan Adherence Failure:** Ocorre quando o agente não segue as diretrizes ou o plano acordado, seja por subexecução, seja pela execução de ações não previstas.
- (2) **Invention of New Information:** Ocorre quando o agente introduz, remove ou altera informação não fundamentada na entrada, no contexto ou na saída das ferramentas, incluindo fabricação de fatos e alucinações.
- (3) **Invalid Invocation:** Ocorre quando o agente emite uma chamada de ferramenta inválida, com entradas malformadas ou argumentos obrigatórios ausentes que falham na validação de esquema.
- (4) **Misinterpretation of Tool Output:** Ocorre quando o agente raciocina incorretamente sobre a saída de uma ferramenta, própria ou de outro agente.
- (5) **Intent-Plan Misalignment:** Ocorre quando o agente interpreta mal o objetivo ou as restrições do usuário e produz um plano incorreto.
- (6) **Under-specified User Intent:** Ocorre quando a tarefa não pode ser concluída por falta de informação completa.
- (7) **Intent Not Supported:** Ocorre quando se solicita uma ação para a qual não há ferramenta disponível.
- (8) **Guardrails Triggered:** Ocorre quando a execução é bloqueada por políticas de segurança ou restrições externas de acesso.
- (9) **System Failure:** Ocorre quando há uma falha de conectividade ao acionar uma ferramenta, como um *endpoint* inacessível [1].

Essas categorias oferecem um vocabulário comum para atribuição de causa raiz e são justamente o alvo de classificação inferido pelo juiz do framework [1].

2.1 Telemetria estruturada e OpenTelemetry

A qualidade do diagnóstico produzido pelo AgentRx depende dos sinais disponíveis na trajetória, e os próprios autores apontam como trabalho futuro a busca por sinais de maior qualidade, capazes de separar verdadeiros positivos de sinalizações ruidosas [1]. Grande parte dos sistemas de agentes registra sua execução por meio de

logs textuais, que tendem a ser heterogêneos e a exigir normalização antes de qualquer análise. A observabilidade de agentes, por sua vez, propõe que a execução seja registrada de forma estruturada, por meio de *traces*, *spans*, eventos e métricas, de modo a viabilizar diagnóstico e responsabilização [2]. Nesse cenário, o OpenTelemetry oferece um vocabulário padronizado para representar execuções observáveis, incluindo atributos de *spans*, relações hierárquicas entre operações, eventos e métricas quantitativas associadas a custo, duração e uso de *tokens* [3].

Neste trabalho, esse vocabulário é empregado para representar cada trajetória do sistema multiagente simulado como uma árvore de *spans*: um *span* raiz envolve a execução completa do *workflow*, e cada agente ou ferramenta é registrado como um *span* filho, indexado pela sua etapa na trajetória. Cada *span* carrega, de forma explícita, o papel do agente e a operação realizada, as mensagens de entrada e saída, e quando aplicável para a ferramenta acionada com seus argumentos e o resultado retornado, além de métricas quantitativas de uso de *tokens* e de duração e do estado de conclusão da operação. Eventos discretos marcam transições relevantes da execução, como a delegação de uma tarefa entre agentes, o planejamento de uma chamada de ferramenta, a falha de uma dependência ou o resultado de uma etapa de validação. Dessa forma, elementos que em um log textual permaneceriam implícitos (quem agiu sobre qual ferramenta, com que resultado, em que ponto da hierarquia e com qual estado) tornam-se campos estruturados e diretamente inspecionáveis.

3 Desenho experimental

3.1 Sistema multiagente simulado

O ambiente foi implementado com LangGraph como um MAS de quatro agentes: **Coordenador**, **Pesquisador**, **Executor** e **Avaliador**, que acessam uma ferramenta de catálogo *mockada* e controlada localmente. Embora sejam quatro agentes, a trajetória instrumentada registra cinco passos, pois a execução da ferramenta é capturada como um passo próprio entre o Pesquisador e o Executor. Seguindo a Figura 1, uma tarefa percorre o sistema assim: (1) a tarefa é definida e enviada ao Coordenador, que interpreta a solicitação e define, de forma determinística, a consulta e o plano, delegando-os ao Pesquisador; (2) o Pesquisador analisa as ferramentas disponíveis e prepara a chamada concreta a partir do plano; (3) a ferramenta *ProductCatalogSearch* é executada, retornando o resultado da busca no catálogo; (4) o Executor sintetiza a resposta a partir desse resultado; e (5) o Avaliador decide, deterministicamente, se há evidência suficiente da ferramenta para considerar a tarefa concluída. As decisões estruturais (plano, argumentos, veredito) são determinísticas; ao LLM cabe apenas reescrever a justificativa textual, o que mantém o comportamento estável entre execuções. O MAS é agnóstico ao modelo do agente; neste estudo, os agentes de execução foram configurados com o *Gemma 3 – 27B*. As ferramentas *mockadas* garantiram reprodutibilidade e permitiram a injeção explícita de falhas.

3.2 Geração dos cenários

Os cenários partiram dos dados de catálogo do *tau-bench*¹, o mesmo *benchmark* de fluxos de API usado no estudo original. As tarefas transacionais do *tau-bench* não foram reutilizadas: são multi-turno e alteram estado, ao passo que o MAS opera em turno único e apenas sobre leitura do catálogo. Reaproveitou-se, então, o catálogo de produtos como insumo de um gerador determinístico, que instanciou consultas de leitura e computou, a partir do próprio catálogo, a resposta correta de cada uma. Foram gerados 30 cenários, distribuídos igualmente (seis por cada categoria) entre as cinco categorias de falha consideradas. Cada cenário originou um par de trajetórias que representa a mesma tarefa e a mesma falha, diferindo apenas na representação.

3.3 Injeção de falhas e *ground truth*

As falhas foram introduzidas por operadores scriptados e determinísticos, aplicados ao passo causal de cada categoria: *Instruction/Plan Adherence Failure* no Coordenador (passo 1), *Invalid Invocation* no Pesquisador (passo 2), *System Failure* na ferramenta (passo 3), e *Misinterpretation of Tool Output* e *Invention of New Information* no Executor (passo 4). Cada cenário recebeu uma única falha, de modo que o *ground truth* foi definido operacionalmente pelo ponto de injeção e pela categoria do operador aplicado. Assim, para as métricas de acurácia da etapa crítica e da categoria, considera-se correta a identificação do passo em que a falha foi introduzida por código e da categoria correspondente. Como a injeção é controlada, esse gabarito não depende da competência do modelo do agente.

Essa definição, contudo, deve ser interpretada dentro do escopo do desenho experimental. Em sistemas multiagentes, o ponto de injeção, o primeiro desvio observável e o ponto em que a execução se torna irrecuperável podem não coincidir. Neste estudo, a avaliação privilegia o ponto de injeção como referência de causa controlada; portanto, quando o juiz aponta um passo posterior associado ao sintoma da falha, essa predição é contabilizada como erro de localização, ainda que possa representar uma interpretação plausível da manifestação observável.

Para mitigar ameaças à validade, o evento *fault injected* e os atributos que revelam a categoria ou o nó injetado foram removidos das trajetórias avaliadas; o juiz não recebeu o *ground truth*; e os dois braços foram derivados do mesmo *trace* OTel bruto. Além disso, o *trace* registra contexto de execução no próprio passo da causa, como o contrato da ferramenta no Pesquisador e a consulta planejada no Coordenador, tornando *Invalid Invocation* e *Instruction/Plan Adherence Failure* observáveis no ponto em que foram introduzidas.

3.4 Representações comparadas e adaptação à IR

De cada execução coletou-se um *trace* bruto em formato OpenTelemetry (JSON), fonte única das duas representações comparadas,

¹<https://github.com/sierra-research/tau-bench>

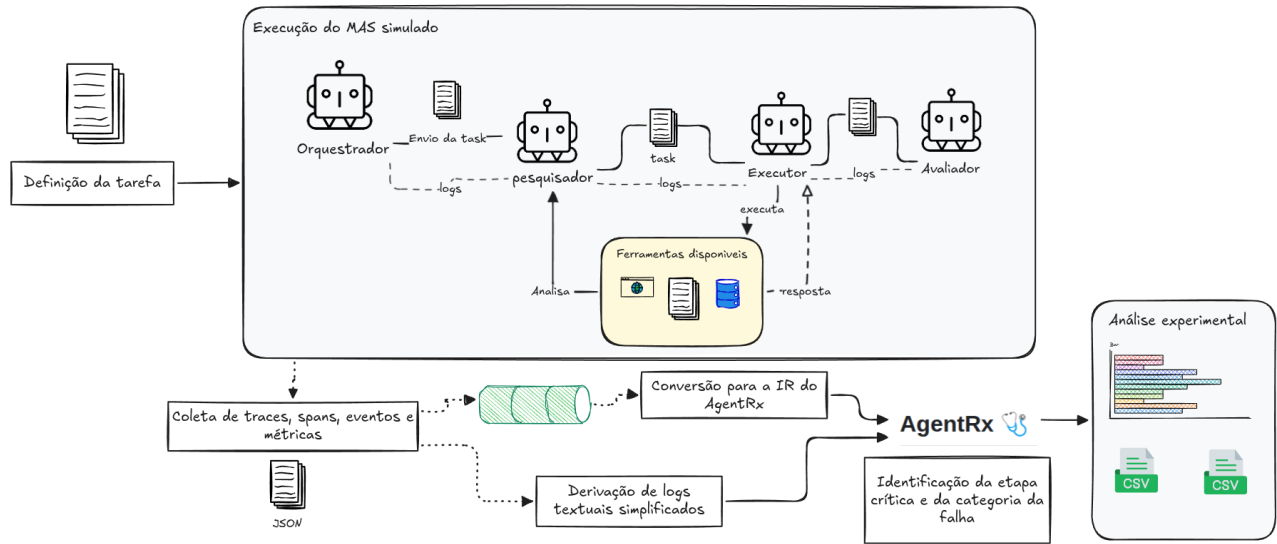


Figura 1: Visão geral do desenho experimental proposto.

ambas julgadas pelo AgentRx. O **braço A (telemetria estruturada)** preserva, em cada passo, papel do agente, operação, mensagens de entrada e saída, ferramenta e argumentos quando aplicável, resultado da ferramenta, métricas de *tokens* e duração, estado de conclusão e relações hierárquicas entre *spans*. O **braço B (log textual simplificado)** descreve a mesma trajetória descritiva, sem os campos de telemetria. Como os dois braços carregam o mesmo conteúdo semântico e diferem apenas na telemetria, a comparação isola o efeito da representação. Um adaptador converteu ambos para a IR do AgentRx; como essa IR representa cada passo por um papel e um conteúdo textual, os elementos de telemetria do braço A foram inseridos no próprio conteúdo do passo, de forma textual.

3.5 Execução do AgentRx e métricas

Utilizou-se o modelo *GPT-5.5* como juiz, listado como modelo padrão na configuração (.env) do repositório do AgentRx² via Copilot CLI. Cada trajetória foi julgada dez vezes, agregando-se por cenário a categoria pela moda e o passo pela média, conforme o estudo original [1]. As métricas reproduzidas foram **Critical Step Accuracy**, **Failure Category Accuracy** e **Average Step Distance** [1], complementadas pela acurácia tolerante em até um passo (Passo ± 1). Como cada cenário possui uma única falha anotada, as acurácias de categoria crítica e *any* coincidem. A comparação entre os braços A e B foi pareada por cenário.

4 Resultados

Esta seção apresenta os resultados do experimento MAS-SIM (agente Gemma 3–27B; juiz GPT-5.5; dez julgamentos por trajetória; comparando o braço A (telemetria estruturada) e o braço B (log textual simplificado) sobre os mesmos 30 cenários pareados. Os valores são preliminares e devem ser lidos como evidência descritiva: como mostra a análise pareada (Seção 4.4), nenhuma diferença entre os braços foi estatisticamente detectável.

²<https://github.com/microsoft/AgentRx>

Tabela 1: Resumo das métricas principais no experimento MAS-SIM.

Métrica	Telemetria (A)	Log textual (B)	$\Delta A-B$
Critical Step Accuracy	83,3%	86,7%	−3,3 p.p.
Passo ± 1	96,7%	96,7%	,0 p.p.
Failure Category Accuracy	86,7%	80,0%	+6,7 p.p.
Average Step Distance	0,263	0,210	+0,053

4.1 Resultados agregados

A Tabela 1 apresenta as métricas principais seguindo a estrutura proposta pelo autores do AgentRx [1]. Na identificação exata da etapa crítica, o log textual simplificado obteve 86,7% de acurácia e a telemetria estruturada 83,3%, uma diferença de −3,3 pontos percentuais para a telemetria, pequena e sem vantagem prática clara. Sob tolerância de um passo (Passo ± 1), ambos os braços empataram em 96,7%. Na classificação da categoria da causa raiz, o comportamento se inverteu: a telemetria alcançou 86,7% contra 80,0% do log textual, uma diferença descritiva de +6,7 pontos. A distância média de passo foi ligeiramente menor no log textual (0,210 contra 0,263). Em síntese, a telemetria não produziu ganho uniforme, exibindo comportamento distinto conforme a dimensão avaliada.

4.2 Localização da etapa crítica

Considerando os cenários individualmente, o log textual localizou corretamente a etapa crítica em 26 dos 30 cenários e a telemetria em 25. As tolerâncias maiores atingiram 100% em ambos os braços (Passo ± 3 e Passo ± 5), o que caracteriza um efeito-teto associado ao tamanho das trajetórias, compostas por cinco passos. A Tabela 2 detalha a distância de passo: em ambos os braços a mediana foi zero, confirmando que a maioria dos cenários foi localizada com exatidão, e a distância média foi ligeiramente menor no log textual

Tabela 2: Estatísticas descritivas da distância de passo no experimento MAS-SIM. Valores menores indicam melhor localização da etapa crítica.

Braço	Média	DP	Med.	Mín.	Máx.	Norm.	MAE
Telemetria (A)	0,263	0,629	0	0	3	0,053	0,263
Log textual (B)	0,210	0,596	0	0	3	0,042	0,237

(0,210 contra 0,263), o que indica uma vantagem descritiva pequena na localização.

4.3 Classificação da categoria da causa raiz

Na classificação da categoria, a telemetria acertou 26 dos 30 cenários e o log textual 24. A Tabela 3 desagrega esse resultado por categoria de falha. Ambos os braços tiveram desempenho perfeito em *System Failure*, *Invalid Invocation* e *Misinterpretation of Tool Output*. A diferença agregada origina-se de uma única categoria, *Invention of New Information*, na qual a telemetria obteve 100,0% e o log textual 66,7%. A categoria *Instruction/Plan Adherence Failure* foi a mais difícil, com 33,3% de acurácia em ambos os braços. A Tabela 4 apresenta a mesma leitura no nível das repetições individuais do juiz (60 julgamentos por categoria em cada braço), evidenciando que os erros de categoria se concentram em *Invention* e, sobretudo, em *Plan Adherence*.

4.4 Comparação pareada

A Tabela 5 apresenta o teste de McNemar para as duas métricas categóricas pareadas. Não houve diferença estatisticamente detectável entre os braços, nem para a classificação da categoria ($p = 0,617$) nem para a localização exata da etapa ($p = 1,000$). As diferenças descritivas observadas nas seções anteriores devem, portanto, ser interpretadas com cautela, dado o tamanho reduzido da amostra e o baixo poder estatístico do teste.

5 Discussão

A discussão retoma as questões de pesquisa formuladas na introdução. Reitera-se que os achados são preliminares: como nenhuma diferença foi estatisticamente detectável (Seção 4.4), as respostas a seguir descrevem tendências, não vantagens estabelecidas.

5.1 RQ1: a telemetria estruturada melhora a identificação da etapa crítica?

Os resultados não sustentam uma resposta afirmativa. O log textual simplificado foi ligeiramente superior à telemetria na localização exata (86,7% contra 83,3%; distância média 0,210 contra 0,263), e os dois braços empataram sob tolerância de um passo (96,7%). Como a diferença não foi estatisticamente detectável ($p = 1,000$) e as trajetórias têm apenas cinco passos, o mais adequado é concluir que, no formato avaliado, a telemetria estruturada **não** melhorou a identificação da etapa crítica em relação ao log textual.

5.2 RQ2: a telemetria estruturada reduz ambiguidades na categorização da causa raiz?

Há indício descritivo nesse sentido, porém fraco. A telemetria obteve maior acurácia de categoria (86,7% contra 80,0%), mas a diferença não foi estatisticamente detectável ($p = 0,617$) e decorre inteiramente de uma única categoria, *Invention of New Information*, na qual a telemetria acertou 100,0% contra 66,7% do log textual (Tabela 3); nas demais categorias em que ambos foram bem, o desempenho foi idêntico. Uma leitura plausível é que a telemetria tornou mais explícita a ausência de evidência da ferramenta no passo em que a resposta foi sintetizada, reduzindo a ambiguidade nesse caso específico. Por depender de uma só categoria e de poucos cenários, o achado é frágil e requer replicação.

5.3 Análise post-hoc dos erros

Além da comparação planejada, inspecionaram-se os cenários em que os braços divergiram, a fim de compreender *por que* o diagnóstico falhou. Essa análise é exploratória e post-hoc.

Em *Invalid Invocation*, o log textual localizou 5 de 6 cenários e a telemetria 4 de 6. A inspeção revela uma ambiguidade entre causa e sintoma: o juiz tende a atribuir a falha ao passo do sintoma (o *SchemaError*, no passo 3) em vez do passo da causa (a chamada mal-formada, no passo 2). A telemetria, por expor mais detalhes do erro lançado, parece amplificar o passo 3 como discriminador — no cenário q07, 9 das 10 repetições do braço A migraram para o passo 3, e o cenário q10 foi errado por ambos pela mesma razão. Como causa e sintoma são passos adjacentes, o empate sob Passo ± 1 mostra que se trata de ambiguidade fina, não de erro grosseiro.

A categoria *Instruction/Plan Adherence Failure* teve o pior desempenho em ambos os braços (33,3% de categoria; 3 de 6 na localização), sem que a telemetria a atenuasse. A falha é injetada no Coordenador (passo 1) como uma consulta que viola silenciosamente uma restrição da tarefa, **sem** gerar erro ou status anômalo em nenhum passo. Os erros refletem esse caráter silencioso: nos cenários q26 e q28 o juiz não detectou falha alguma (veredito *Inconclusive*, passo 0, em 10 de 10 repetições); em q30 detectou a resposta incorreta, mas a atribuiu ao Executor como *Misinterpretation* (passo 4); e, mesmo quando o passo 1 foi corretamente localizado (q25 e q27), a categoria oscilou entre *Instruction/Plan Adherence Failure* e a categoria vizinha *Intent-Plan Misalignment*. Esse padrão sugere um limite da telemetria operacional: sinais como *spans*, status, erro, duração, *tokens* e chamadas de ferramenta capturam mal falhas de aderência ao plano, pois estas não se manifestam como anomalia operacional, mas como divergência entre o plano e a intenção da tarefa. Diagnosticá-las poderia exigir uma camada de telemetria *semântica* — registrando, por exemplo, o plano esperado, o subobjetivo ativo, as restrições da tarefa, a ação esperada *versus* a executada e a justificativa do agente. Essa é a direção mais promissora aberta pelo estudo.

6 Conclusão

Este trabalho apresentou uma replicação controlada do AgentRx em um sistema multiagente simulado, com o objetivo de investigar se a forma de representação da trajetória de execução afeta o diagnóstico de falhas. Para isso, compararam-se duas representações da

Tabela 3: Resultados por categoria de falha no experimento MAS-SIM.

Categoria	Braço	Failure Category Accuracy	Critical Step Accuracy	Average Step Distance
System Failure	Telemetria (A)	100,0%	6/6	0,000
System Failure	Log textual (B)	100,0%	6/6	0,000
Invalid Invocation	Telemetria (A)	100,0%	4/6	0,417
Invalid Invocation	Log textual (B)	100,0%	5/6	0,217
Misinterpretation of Tool Output	Telemetria (A)	100,0%	6/6	0,000
Misinterpretation of Tool Output	Log textual (B)	100,0%	6/6	0,000
Invention of New Information	Telemetria (A)	100,0%	6/6	0,000
Invention of New Information	Log textual (B)	66,7%	6/6	0,000
Instruction/Plan Adherence Failure	Telemetria (A)	33,3%	3/6	0,900
Instruction/Plan Adherence Failure	Log textual (B)	33,3%	3/6	0,833

Tabela 4: Frequência de acertos/erros nas repetições do juiz por categoria de falha no experimento MAS-SIM. Cada célula de frequência apresenta acertos/erros em 60 julgamentos por braço (6 cenários × 10 repetições).

Categoria	Cat. A	Cat. B	Passo A	Passo B	MAE A	MAE B
System Failure	60/0	60/0	60/0	60/0	0,000	0,000
Invalid Invocation	60/0	60/0	35/25	47/13	0,417	0,217
Misinterpretation of Tool Output	60/0	60/0	60/0	60/0	0,000	0,000
Invention of New Information	51/9	47/13	60/0	60/0	0,000	0,000
Instruction/Plan Adherence Failure	23/37	21/39	30/30	29/31	0,900	0,967

Tabela 5: Teste de McNemar para as métricas categóricas pareadas no experimento MAS-SIM.

Métrica	Ambos	Só A	Só B	Nenhum	<i>p</i>
Failure Category Accuracy	23	3	1	3	0,617
Critical Step Accuracy	25	0	1	4	1,000

mesma trajetória controlada — telemetria estruturada (braço A) e log textual simplificado (braço B) —, ambas processadas pelo componente de julgamento do AgentRx, sobre 30 cenários pareados com falhas injetadas em cinco categorias.

Os resultados são preliminares e devem ser lidos como evidência descritiva, uma vez que o teste de McNemar não detectou diferença estatisticamente significativa entre os braços em nenhuma das métricas ($p = 0,617$ para a categoria e $p = 1,000$ para a localização). Quanto à **RQ1**, a telemetria estruturada não melhorou a identificação da etapa crítica: o log textual simplificado foi ligeiramente superior na localização exata e na distância média, com empate sob tolerância de um passo. Quanto à **RQ2**, observou-se um indício descritivo, porém fraco, de que a telemetria reduz ambiguidades na categorização da causa raiz; a vantagem, contudo, decorreu inteiramente de uma única categoria, *Invention of New Information*, e não foi estatisticamente detectável. O principal achado, portanto, é que a telemetria estruturada, no formato avaliado, não produziu ganho robusto e generalizado sobre a representação textual, mas pode contribuir para reduzir ambiguidades em categorias específicas de falha.

A análise post-hoc dos erros trouxe as contribuições mais informativas do estudo. A categoria *Instruction/Plan Adherence Failure* foi a mais difícil para ambos os braços, e a inspeção dos cenários sugere que falhas de aderência ao plano, por não se manifestarem como anomalia operacional, são mal capturadas pela telemetria operacional comum. Esse achado indica que sinais como *spans*, status, erros, duração e chamadas de ferramenta podem ser insuficientes para esse tipo de falha, apontando a necessidade de uma camada de telemetria *semântica* — que registre, por exemplo, o plano esperado, as restrições da tarefa e a ação esperada *versus* a executada.

Como trabalho futuro, pretende-se ampliar o número de cenários e empregar trajetórias mais longas e complexas — com *retries*, múltiplas chamadas de ferramenta e falhas encadeadas —, capazes de mitigar o efeito-teto observado nas trajetórias curtas. Pretende-se, ainda, avaliar o método sobre *logs* naturais de sistemas multiagentes reais, em contraste com as representações derivadas de uma mesma fonte controlada, e investigar quais sinais específicos de telemetria — operacionais ou semânticos — contribuem para diagnósticos mais precisos.

7 Ameaças à Validade

Validade interna. O processo de geração das trajetórias, o adaptador para a IR, a agregação dos julgamentos e a configuração do modelo juiz podem influenciar os resultados. Para reduzir esse risco, a injeção foi *scriptada* e determinística, o evento `fault.injected` e os atributos que revelam a falha foram removidos das trajetórias,

o juiz não recebeu o *ground truth* e os dois braços foram derivados do mesmo *trace* bruto.

Validade de construto. Duas limitações merecem destaque. Primeiro, os dois braços são **duas representações de uma mesma fonte controlada**: o log textual simplificado é derivado da telemetria, e não coletado de um sistema real. Portanto, o estudo compara representações alternativas da mesma trajetória, uma com telemetria estruturada e outra textual, e **não** logs naturais heterogêneos contra telemetria estruturada. Segundo, a IR do AgentRx representa cada passo apenas por papel e conteúdo textual; assim, a telemetria do braço A entra como texto, e não como campos estruturados, o que pode subaproveitar seu potencial. Além disso, o *ground truth* adota o ponto de injeção como causa, que pode não coincidir com o passo em que a execução se torna irrecuperável.

Validade externa. O estudo usa um MAS simulado, tarefas sintéticas, ferramentas *mockadas* e apenas 30 cenários, com trajetórias curtas (cinco passos). As trajetórias curtas produzem efeito-teto nas tolerâncias Passo ± 3 e ± 5 (100% em ambos os braços), que

deixam de diferenciar os métodos. Os resultados, portanto, não se generalizam diretamente para sistemas reais. Trabalhos futuros devem empregar trajetórias mais longas e complexas, múltiplas chamadas de ferramenta, falhas recuperáveis anteriores à falha principal e erros que emergem por encadeamento.

Validade de conclusão. A amostra é pequena e os testes têm baixo poder estatístico; o teste de McNemar não detectou diferença significativa em nenhuma métrica ($p = 0,617$ e $p = 1,000$). Os resultados devem ser interpretados como **preliminares e exploratórios**, indicando tendências a confirmar em estudos maiores.

Referências

- [1] Shraddha Barke, Arnav Goyal, Alind Khare, Avaljot Singh, Suman Nath, and Chetan Bansal. 2026. AgentRx: Diagnosing AI Agent Failures from Execution Trajectories. *arXiv preprint arXiv:2602.02475* (2026).
- [2] Liming Dong, Qinghua Lu, and Liming Zhu. 2024. Agentops: Enabling observability of llm agents. *arXiv preprint arXiv:2411.05285* (2024).
- [3] IBM. 2024. O que é OpenTelemetry? <https://www.ibm.com/br-pt/topics/opentelemetry>. Acessado em: 02 jun. 2026.